CIND820 Data Preparation

November 10, 2026

0.1 CIND820 Data Preparation

The raw data from QMJHL game centres must undergo numerous calculations and transformations to prepare the variables relevant to this data investigation. The variables for the expected goals (xG) models are as follows:

- Shot distance
- Shot angle
- Time since last shot
- Distance from last shot
- Current shot coordinates
- Previous shot coordinates
- Time elapsed in game
- Player strength
- Score differential

This will be built off the raw API from the QMJHL Game Centre pages.

[1]:
```
import pandas as pd

raw = pd.read_csv('2026-27 QMJHL PBP.csv')
```

There is a large number of columns that can be removed from this dataset given our intended use of the data. Any rows where 'event' = 'goalie_change' will also be deleted, since these data points are not needed because the necessary information is provided in other columns of the remaining dataset.

[2]:
```
new = raw[['event', 'period_id', 's', 'team_id', 'x_location', 'y_location',␣
↪'home', 'game_goal_id', 'player_team_id', 'goalie_team_id', 'player',␣
↪'goalie', 'player_served', 'pp', 'penalty_shot', 'minutes',␣
↪'goal_type_name', 'game_id']]

new = new[new["event"] != "goalie_change"]
```

There are a number logical steps that must be considered when cleaning this data for use.

Duplicated goal events Every goal is entered as two separate events, a 'shot' and a 'goal' (since a goal always comes from a shot). We only want to keep one copy of these events to avoid goals being overrepresented/duplicated and skewing the dataset.

Crucially, various important variables are not populated in the 'goal' event ('player_team_id,' 'goalie_team_id,' 'player,' and 'goalie'). Therefore, we are going to remove the 'goal' events from the initial dataset, then reclassify shots that have a 'game_goal_id' as goals.

[3]:
```
new = new[new["event"] != "goal"]

new.loc[new['game_goal_id'].notna(), 'event'] = 'goal'
```

Removing incidental penalties Sometimes, penalties "cancel out" (if an equal number of players on each team are penalized for the same actions, like roughing/fighting). This dataset labels these types of penalties helpfully, where 'pp' (power play) = 0. We do not need the information for these penalties in our analysis (we are keeping track of penalties to identify how many players are on the ice for each team). If 'pp' = 0, that means that the offending team does not lose a player on the ice. Therefore, we can drop every event = 'penalty' entry where 'pp' = 0.

[4]:
```
new = new.drop(new[new['pp'] == 0].index)
```

Score differential One of the variables that will be used to build the model is score differential. This integer will represent the status of the game's score when the shot occurred, such as when the game is tied (score_diff = 0), the shooting team is down by two (score_diff = -2), or the shooting team is up by three (score_diff = 3). A new column called 'goal_change' will help to calculate this by expressing 'goal' events as positive or negative integers. A goal for the home team will be represented by 1, and a goal for the away team will be represented by -1.

Using the .cumsum() (cumulative sum) function in the 'score_diff' column, calculated from the 'goal_change' values, we can fill the 'score_diff' column with an integer representing the shooting team's game situation. We'll also use the .shift(1, fill_value=0) function to ensure that the shot that becomes the goal illustrates the game situation when the shot was taken (for example, we still want the first goal of the game to read 0 in the 'score_diff' column to show that the game was tied when the shot was taken). The .groupby('game_id') function is also used since our dataset contains multiple games in one csv file.

Finally, np.where() allows us to invert the shot differential for shots taken by the away team. For example, if the home team has two more goals than the away team, 'score_diff' needs to read 2 for shots where 'home' = 1 and -2 for shots where 'home' = 0.

[5]:
```
import numpy as np

new['score_diff'] = None

new['goal_change'] = 0

new.loc[(new['event'] == 'goal') & (new['home'] == 1), 'goal_change'] = 1
new.loc[(new['event'] == 'goal') & (new['home'] == 0), 'goal_change'] = -1

new['score_diff'] = new.groupby('game_id')['goal_change'].cumsum().shift(1,␣
↪fill_value=0)
```

```
new['score_diff'] = np.where(new['home'] == 0, -new['score_diff'],␣
↪new['score_diff'])
```

Time elapsed in game In the original dataset, the 's' variable (expressing time elapsed in seconds) only expresses the time elapsed in each period, not the entire game. Because we want to analyze the timing of events in the game as a whole (looking at early/late goals, for example), 1200 seconds (20 minutes) will be added to any 's' value where 'period_id' = 2. 2400 seconds will be added to 's' values where 'period_id' = 3, and 3600 seconds will be added to 's' values where 'period_id' = 4.

[6]:
```
new['s'] = np.where(new['period_id'] == 2, new['s'] + 1200, new['s'])
new['s'] = np.where(new['period_id'] == 3, new['s'] + 2400, new['s'])
new['s'] = np.where(new['period_id'] == 4, new['s'] + 3600, new['s'])
```

Play strength classification Penalties have remained in the dataset thus far because they provide crucial information that will be important in the analysis. Because teams generally generate more shots and goals on the power play (5-on-4) because they have more players on the ice than the other team, this is an important variable to consider. Having this data can help the model identify shots and goals that are more likely to go in because the defending team is at a disadvantage.

One preliminary consideration in the data is that double-minor (four-minute-long) penalties are inputted as two penalties of two minutes each. We want to transform these into one four-minute-long penalty entry, especially because if the other team scores during a four-minute power play, their power play continues (as opposed to a two-minute power play, where it ends after the goal).

This is done by making a mask using .duplicated() with a subset of 's', 'game_id', and 'player_served' - finding duplicates where the game, time of the event, and the player ID of who's serving the penalty are identical (double-minor penalties are served to the same player; we need to include this player ID in the subset because sometimes multiple different players will get penalties at the same time). The double_minors DataFrame made from these conditions will adopt a 'minutes' value of 4.0 before being incorporated back into the main new DataFrame. From the new DataFrame, the duplicated events will be dropped (the second copy of the double_minors where 'minutes' = 4.0, as well as the original penalties where 'minutes' = 2.0). These double_minors events are reintegrated into the dataset in their correct original order.

[7]:
```
double_minor_mask = new.duplicated(subset=['s', 'game_id', 'player_served'],␣
↪keep=False)

double_minors = new[double_minor_mask & (new['event'] == 'penalty')]

double_minors['minutes'] = 4.0

new.loc[double_minors.index, :] = double_minors[:]

new = new.drop_duplicates(subset=['s', 'game_id', 'player_served'],␣
↪keep='first')
```

```
/var/folders/0n/31k33nxn59sgvdqjp51nfl2c0000gn/T/ipykernel_2871/3438194613.py:5:
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  double_minors['minutes'] = 4.0
```

Below are two functions filling out the new 'home_strength' and 'away_strength' variables to keep track of how many players for each team are on the ice for any given event. We want to keep track of the number of penalties on either team because it affects how many players are on the ice for each team, which affects shots and goals (a power play, having more players on the ice than the other team, often leads to more goals).

This user-defined function takes 'event,' 's,' 'minutes,' and 'home.' into consideration while being able to store active penalties and their start and end times to label the strengh (number of players on the ice) during an event. In calc_home_strength, when event = 'penalty' and home = 1, the 's' value in that row is appended to the active_penalties list, as well as the end time ('s' + 'minutes' * 60, to convert 'minutes' from minutes into seconds). active_count dictates the number that will be printed in the 'home_strength' row, which will be a positive integer before further data transformation.

We also need to be able to end minor penalties if a goal is scored, but not double-minors or major (5 minutes) penalties. In the calc_home_strength function, this loop occurs if event = 'goal' and active_count > 0 and home $= 0$ (if the away team scores). If there is an active penalty and its duration is 120 seconds (2 minutes), then the end time of the penalty is overwritten as the time of the goal, because this is the situation where a goal for the other team ends the penalty.

[8]:
```
new['home_strength'] = np.nan
new['away_strength'] = np.nan

def calc_home_strength(new):
    strengths = []
    active_penalties = []

    for idx, row in new.iterrows():
        event = row['event']
        time = row['s']
        duration = row['minutes'] * 60 # converted from minutes (2.0) to␣
 ↪seconds (120.0) to match 's' format
        home = row['home']

        active_penalties = [p for p in active_penalties if p[1] > time]

        if event == 'penalty' and home == 1:
            active_penalties.append([time, time + duration])
            active_penalties.sort(key=lambda x: x[0])

        active_count = sum(1 for start, end in active_penalties if start <=␣
 ↪time < end)
```

```
        row_strength = active_count

        # If a goal is scored, end the oldest active minor penalty. Only one␣
 ↪penalty gets ended at a time by a goal (for example, if a team scores at␣
 ↪5v3, one player is released and they then play 5v4).
        if event == 'goal' and active_count > 0 and home == 0:
            for p in active_penalties:
                if p[0] <= time < p[1]  and (p[1] - p[0] == 120.0):
                    p[1] = time
                    break
            active_penalties = [p for p in active_penalties if p[1] > time]

        strengths.append(row_strength)

    new['home_strength'] = strengths
    return new

new = calc_home_strength(new)

def calc_away_strength(new):
    strengths = []
    active_penalties = []

    for idx, row in new.iterrows():
        event = row['event']
        time = row['s']
        duration = row['minutes'] * 60 # converted from minutes (2.0) to␣
 ↪seconds (120.0) to match 's' format
        home = row['home']

        active_penalties = [p for p in active_penalties if p[1] > time]

        if event == 'penalty' and home == 0:
            active_penalties.append([time, time + duration])
            active_penalties.sort(key=lambda x: x[0])

        active_count = sum(1 for start, end in active_penalties if start <=␣
 ↪time < end)

        row_strength = active_count

        # If a goal is scored, end the oldest active minor penalty. Only one␣
 ↪penalty gets ended at a time by a goal (for example, if a team scores at␣
 ↪5v3, one player is released and they then play 5v4).
        if event == 'goal' and active_count > 0 and home == 1:
            for p in active_penalties:
```

```
                if p[0] <= time < p[1] and (p[1] - p[0] == 120.0):
                    p[1] = time
                    break
            active_penalties = [p for p in active_penalties if p[1] > time]

        strengths.append(row_strength)

    new['away_strength'] = strengths
    return new

new = calc_away_strength(new)
```

After these functions, the outputs are standardized to communicate team strength in terms of how many players each team has on the ice. However, we need to consider a special case when period_id = 4. In overtime, which is regularly played at 3v3 instead of 5v5, if a team has a penalty, the other team will play at a 4v3 advantage (instead of 3v2).

After this step, these numbers can be combined into a object with a format such as '5v5.' This is ordered so that the number of players on the team taking the shot or scoring the goal comes first. We need to use different calculations for regulation time vs. overtime. To avoid complicating the defined function above, this is the step where the home_strength and away_strength will be processed.

Example: If the home team gets a penalty with 60 seconds left in the third period, a shot they take with 30 seconds left in the third period will have its strength calculated as [negative(number of home team penalties) + 5] v [negative(number of away team penalties) + 5] = 4v5. In overtime, this is calculated as [number of away team penalties + 3] v [number of home team penalties + 3] = 3v4. The away team in this scenario gets an extra player because they have a power play. The lines below ensure that overtime strength is expressed accurately.

[9]:
```
str_conds = [
    (new['home'] == 1) & (new['period_id'] != 4),
    (new['home'] == 0) & (new['period_id'] != 4),
    (new['home'] == 1) & (new['period_id'] == 4),
    (new['home'] == 0) & (new['period_id'] == 4)
]

str_calcs = [
    (-new['home_strength'].shift(1, fill_value=0).abs() + 5).astype(str) + "v"␣
 ↪+ (-new['away_strength'].shift(1, fill_value=0).abs() + 5).astype(str),
    (-new['away_strength'].shift(1, fill_value=0).abs() + 5).astype(str) + "v"␣
 ↪+ (-new['home_strength'].shift(1, fill_value=0).abs() + 5).astype(str),
    (new['away_strength'].shift(1, fill_value=0) + 3).astype(str) + "v" +␣
 ↪(new['home_strength'].shift(1, fill_value=0) + 3).astype(str),
    (new['home_strength'].shift(1, fill_value=0) + 3).astype(str) + "v" +␣
 ↪(new['away_strength'].shift(1, fill_value=0) + 3).astype(str)
]
```

```
new['strength'] = np.select(str_conds, str_calcs, default=np.nan)
```

Empty net goals We don't want to include empty net shots or goals (when the losing team pulls their goalie for a sixth skater to increase their chances of scoring a goal) in the actual analysis. We still need to wait to delete them until after the score differential and team strength is calculated, as it is possible for a team to score after an empty net goal and we want this to be reflected in the dataset. It is also possible for these goals to happen on the power play, which means the penalty would end with that goal being scored.

A shot on goal is defined as when the goalie has to make a save to prevent the puck from entering the net; therefore every shot recorded on an empty net is a goal. The 'goal_type' and 'goal_type_name' variables for both the 'shot' and 'goal' entry of the event will have 'EN' in these cases. We want to exclude these goals entirely from our analysis. It should not affect our game prediction model, as virtually every empty-net goal is for the team that is already winning.

[10]:
```
new = new[~new["goal_type_name"].str.contains("EN", na=False)]
```

Formatting player and goalie names The 'player' and 'goalie' variables in the raw dataset are fully encoded in JSON format like the example below:

- {'player_id': '23329', 'jersey_number': '19', 'team_id': '8', 'team_code': 'SNB', 'first_name': 'Alexis', 'last_name': 'Joseph'}

We want this example above to read as just 'Alexis Joseph.' .str.extract() can help with that, using regular expression (Regex). In this case, we are extracting and combining what follows "'first_name':" and "'last_name':," with a space in between. The enclosed capturing group, (.*?), captures all of the text in these single quotation marks with the period and the asterisk, then the question mark ensures that it stops after the first match (a comma after the first name, and a bracket after the last name).

Regex documentation/cheat sheet: https://coderpad.io/regular-expression-cheat-sheet/

It is not possible with this dataset to unilaterally strip the single quotation mark in this first step, because certain names use double quotations to handle names with apostrophes. Take a look at this real example from the dataset to see how double quotations are sometimes used in these cases:

- {'player_id': '23331', 'jersey_number': '77', 'team_id': '5', 'team_code': 'Hal', 'first_name': 'Malik', 'last_name': **"L'Italien"**}

[11]:
```
new["player"] = (
    new["player"].str.extract(r"'first_name': (.*?),")
    + " "
    + new["player"].str.extract(r"'last_name': (.*?)}")
)

new["goalie"] = (
    new["goalie"].str.extract(r"'first_name': (.*?),")
    + " "
    + new["goalie"].str.extract(r"'last_name': (.*?)}")
)
```

The outputs following this initial step follow the format below: * 'Alexis' 'Joseph' * 'Malik' "L'Italien"

After this, we run through both the 'player' and 'goalie' columns again to strip any single or double quotations from the beginning (.str.lstrip()) or end (.str.rstrip()) of these name outputs. Then, to make sure that apostrophes in names are not deleted, .str.replace() is used to circulate through removing single or doubles quotations that are preceded or succeeded by a space. After this, our example outputs will be properly formatted like so: * Alexis Joseph * Malik L'Italien

This information is needed in the dataset because the model can be used to evaluate individual performance. In particular, Question 2 will deal in using the models to identify players to trade for.

[12]:
```
# Strip quotations at beginning
new['player'] = new['player'].str.lstrip('\'"')
new['goalie'] = new['goalie'].str.lstrip('\'"')
# Strip quotations at end
new['player'] = new['player'].str.rstrip('\'"')
new['goalie'] = new['goalie'].str.rstrip('\'"')
# Strip quotations in middle
new['player'] = new['player'].str.replace("' ", " ")
new['player'] = new['player'].str.replace(" '", " ")
new['goalie'] = new['goalie'].str.replace("' ", " ")
new['goalie'] = new['goalie'].str.replace(" '", " ")
new['player'] = new['player'].str.replace('" ', " ")
new['player'] = new['player'].str.replace(' "', " ")
new['goalie'] = new['goalie'].str.replace('" ', " ")
new['goalie'] = new['goalie'].str.replace(' "', " ")
```

Replacing team IDs with team names The 'team_id,' 'player_team_id,' and 'goalie_team_id' variables all take integers to encode team names more simply. Below lists the team IDs as assigned by the API, matched with their team names.

- 1 = Moncton Wildcats
- 2 = Newfoundland Regiment
- 3 = Cape Breton Eagles
- 5 = Halifax Mooseheads
- 7 = Charlottetown Islanders
- 8 = Saint John Sea Dogs
- 9 = Quebec Remparts
- 10 = Chicoutimi Sagueneens
- 11 = Rouyn-Noranda Huskies
- 12 = Gatineau Olympiques
- 13 = Shawinigan Cataractes
- 14 = Drummondville Voltigeurs
- 15 = Val-d'Or Foreurs
- 16 = Baie-Comeau Drakkar
- 17 = Victoriaville Tigres
- 18 = Rimouski Oceanic

- 19 = Blainville-Boisbriand Armada
- 60 = Sherbrooke Phoenix

This information is needed in the dataset to be able to evaluate teams' performance within the models, which will be used to create the win prediction model.

[13]:
```
teamdict = {
    1: "Moncton",
    2: "Newfoundland",
    3: "Cape Breton",
    5: "Halifax",
    7: "Charlottetown",
    8: "Saint John",
    9: "Quebec",
    10: "Chicoutimi",
    11: "Rouyn-Noranda",
    12: "Gatineau",
    13: "Shawinigan",
    14: "Drummondville",
    15: "Val-d'Or",
    16: "Baie-Comeau",
    17: "Victoriaville",
    18: "Rimouski",
    19: "Blainville-Boisbriand",
    60: "Sherbrooke"
}

new["team_id"] = new["team_id"].replace(teamdict)
new["player_team_id"] = new["player_team_id"].replace(teamdict)
new["goalie_team_id"] = new["goalie_team_id"].replace(teamdict)
```

Removing data no longer needed Before moving onto the final calculated fields, we can simplify the dataset by removing some of the rows and columns previously needed to calculate the calculated fields above.

There is one penalty shot goal in the dataset, which doesn't have its coordinates tracked. Because penalty shots and shootouts are unique 1v0 shooting situations, we will drop this from the dataset. Younggren & Younggren similarly excluded penalty shots and penalty shot goals from their NHL regression model. The score status remains encoded.

Penalties no longer need to be included because we have already calculated the strength (number of players on the ice) for each shot event.

[14]:
```
new = new[new['goal_type_name'] != 'PS']
new = new.drop(columns=['team_id', 'game_goal_id', 'player_served', 'pp',␣
 ↪'penalty_shot', 'minutes', 'goal_type_name', 'goal_change', 'home_strength',␣
 ↪'away_strength'])
new = new[new['event'] != 'penalty']
new = new[new['event'] != 'penaltyshot']
```

Time since previous shot One of the key variables featured in NHL-level analysis is time since previous shot. This is because 'flurries' of shots that are close together are considered dangerous chances with a higher likelihood of a goal. Even sustained pressure of 5 to 20 seconds between shots can increase the chance of a goal, as well.

When we calculate this, we want to make sure that times from previous games and periods will not be included in the analysis, therefore we will use the .groupby() function to calculate each game and period separate of each other. Now that we just removed penalty events from the dataset, the dataset now only contains shots and goals. Therefore, we don't need to filter out penalties.

[15]:
```
new['s_since_prev'] = new.groupby(['game_id', 'period_id'])['s'].diff()
```

Converting coordinates to ft The QMJHL API uses an arbitrary measurement of 600 units by 300 units for the ice surface. We want to fit this to the North American standard ice surface size of 200 ft to 85 ft. This step must be taken here to accurately calculate the rest of the fields, which relate to shot locations.

[16]:
```
new['x_location'] = new['x_location'] / 3 # (600 / 200 = 3)
new['y_location'] = new['y_location'] / (300 / 85)
```

Previous shot coordinates We want to carry down the x and y coordinates of the previous shot because this is another factor that can affect the likelihood of a goal. Just like how the coordinates of the current shot can affect the likelihood of a goal, this dimension could bring some clarity to shots that are close together in time, as mentioned just above.

[17]:
```
new['x_prev'] = new.groupby(['game_id', 'period_id'])['x_location'].shift(1)
new['y_prev'] = new.groupby(['game_id', 'period_id'])['y_location'].shift(1)
```

Distance from previous shot The fourth-strongest variable in Younggren & Younggren's 2021 analysis on NHL data, the previous shot coordinates just populated can also be used to measure the distance between two shooting locations. This could have a strong impact on this model as well, as a large value in this variable alongside a small value in the 's_since_prev' variable creates typically dangerous scoring chances. This metric will be measured using the Euclidean distance formula.

[18]:
```
new['distance_prev'] = np.sqrt((new['x_location'] - new['x_prev']) ** 2 +␣
↪(new['y_location'] - new['y_prev']) ** 2)
```

Shot location standardization We have adjusted 'x_location' and 'y_location' to reflect the ice surface's real dimensions as 200 ft by 85 ft. Upon further investigation, the data shows that the away team (where 'home' = 0)'s attacking zone is found at x_location = 100 to 200. It is important to normalize these results relative to each other for direct comparison between home and away games for every team. This is done by subtracting the x_location from 200 and subtracting the y_location from 85 whenever 'home' = 0.

[19]:
```

```

```
new['x_location'] = np.where((new['home'] == 0), 200 - new['x_location'],␣
 ↪new['x_location'])
new['y_location'] = np.where((new['home'] == 0), 85 - new['y_location'],␣
 ↪new['y_location'])
```

We also need to manage the x_prev and y_prev variables. When the previous shot was from the same team, the values do not need to change. However, when the previous shot was from the other team, the coordinate values should change to reflect the distance between shots accurately.

[20]:
```
new['x_prev'] = np.where(new['home'] == new['home'].shift(1), new['x_prev'],␣
 ↪200 - new['x_prev'])
new['y_prev'] = np.where(new['home'] == new['home'].shift(1), new['y_prev'], 85␣
 ↪- new['y_prev'])
```

Shot distance As mentioned in the Interim Report, the net's location on the 600 unit x 300 unit plane used in the API to track shot locations is (33, 150). Converted to ft, this is (11, 42.5). The Euclidean distance formula will be used to calculate shot distance from the net, using the dataset's arbitrary unit of measurement.

[21]:
```
net_location = (11, 42.5)

new['shot_distance'] = np.sqrt((new['x_location'] - net_location[0]) ** 2 +␣
 ↪(new['y_location'] - net_location[1]) ** 2)
```

Shot angle The .arctan2 and .degrees mathematical functions in Numpy will be used to calculate the angle between the location coordinates of the shot and the net's location. The values are also standardized as absolute values. When shot_angle = 0, this means that the shot was taken directly in front of the net, which is generally considered the most dangerous shooting position.

[22]:
```
new['shot_angle'] = (np.degrees(np.arctan2((new['y_location'] -␣
 ↪net_location[1]), (new['x_location'] - net_location[0])))).abs()
```

[23]:
```
new = new.reset_index(drop=True)
```

[24]:
```
new.to_csv('2026-27 QMJHL PBP prepared.csv', index=False)
```

CIND820 Data Analysis

November 10, 2026

0.1 CIND820 Data Analysis

[6]:
```
import pandas as pd

new = pd.read_csv('2026-27 QMJHL PBP prepared.csv')
```

[7]:
```
new.describe()
```

[7]:

	period_id	s	x_location	y_location	home	\
count	9431.000000	9431.000000	9431.000000	9431.000000	9431.000000	
mean	2.022267	1829.237196	41.190613	43.092684	0.517018	
std	0.835539	1040.649928	22.339350	18.095789	0.499737	
min	1.000000	8.000000	2.333333	0.850000	0.000000	
25%	1.000000	938.500000	24.333333	30.883333	0.000000	
50%	2.000000	1832.000000	36.000000	42.500000	1.000000	
75%	3.000000	2720.000000	55.000000	55.250000	1.000000	
max	4.000000	3899.000000	197.000000	85.000000	1.000000	

	game_id	score_diff	s_since_prev	x_prev	y_prev	\
count	9431.000000	9431.000000	8912.00000	8912.000000	8912.000000	
mean	32925.551373	-0.031916	58.02996	98.171716	42.442106	
std	170.765831	1.950158	58.03096	62.865971	18.104641	
min	32651.000000	-11.000000	1.00000	3.000000	0.000000	
25%	32782.000000	-1.000000	17.00000	35.000000	30.316667	
50%	32915.000000	0.000000	40.00000	72.000000	42.500000	
75%	33073.000000	1.000000	80.00000	163.666667	54.400000	
max	33208.000000	11.000000	604.00000	197.666667	84.150000	

	distance_prev	shot_distance	shot_angle
count	8912.000000	9431.000000	9431.000000
mean	66.654598	34.989275	28.037638
std	49.606158	22.672583	18.661977
min	0.437480	3.253033	0.000000
25%	24.240899	17.231516	13.365964
50%	48.124175	30.802038	26.243297
75%	114.866699	49.410975	38.756723
max	183.172244	187.626284	116.631315

The DataFrame description table above shows the counts, means, standard deviations, and ranges

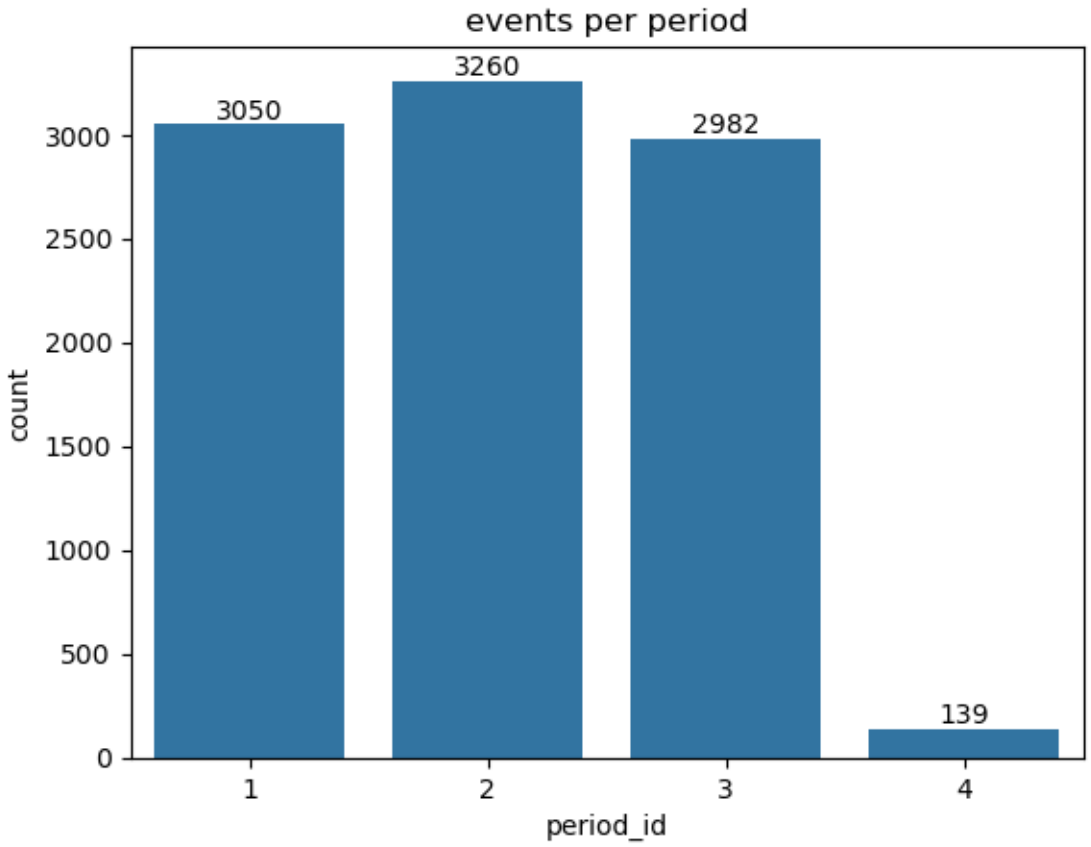
for all of the numerical variables in the dataset. The 'count' row shows that for most variables, the columns are fully populated. The exceptions are 's_since_prev,' 'x_prev,' 'y_prev,' and 'distance_prev' because they take a 'NaN' value when they are the first shot of the period. Therefore, there are 519 periods of data in this dataset (of games up to November 9th, 2026).

period_id

[8]:
```
import matplotlib.pyplot as plt
import seaborn as sns

per_count = sns.countplot(data=new, x='period_id')
per_chart = per_count.bar_label(per_count.containers[0])

plt.title('events per period')
plt.show(per_chart)
```

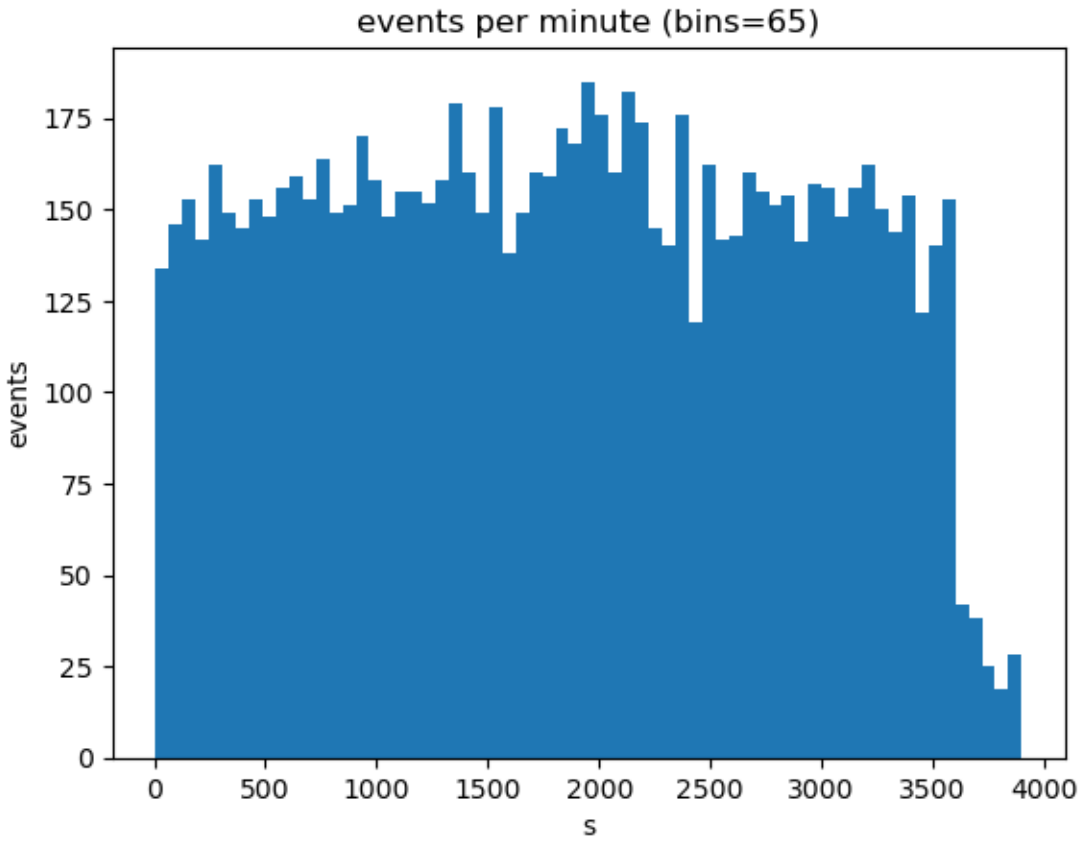


The bar chart above shows the distribution of events by period. The second period has the most number of shot events, followed by the first period and then very closely by the third period. There are remarkably fewer events where 'period_id' = 4 because overtime is only played if the score is still tied at the end of the third period, plus the overtime period itself is only five minutes long (as opposed to the other three periods being 20 minutes long).

s

[27]:
```
s_chart = plt.hist(data=new, x='s', bins=65)

plt.title('events per minute (bins=65)')
plt.ylabel('events')
plt.xlabel('s')
plt.show(s_chart)
```



The histogram above shows the number of events at each approximate minute-long (60-second) interval.

x_location and y_location

[29]:
```
import matplotlib.patches as patches

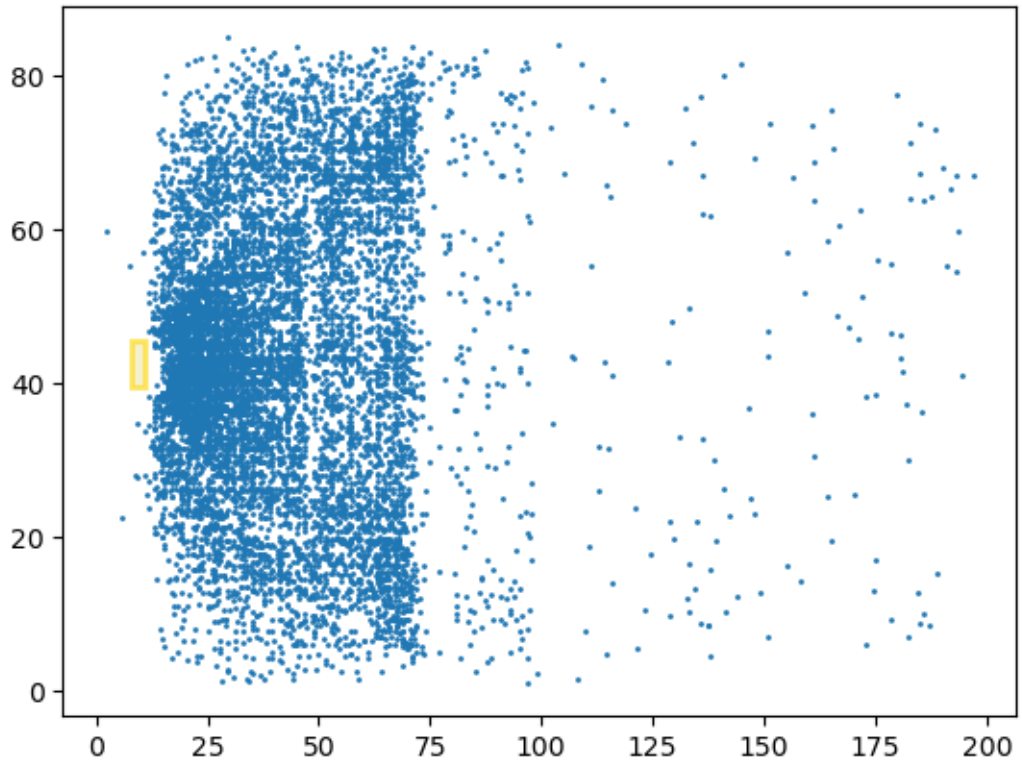
fig, ax = plt.subplots()

shot_scatter = ax.scatter(data=new, x='x_location', y='y_location', s=1)

net = patches.Rectangle((8, 39.5), 3, 6, facecolor='palegoldenrod',␣
 ↪edgecolor='gold', linewidth=2, alpha=0.6)
```

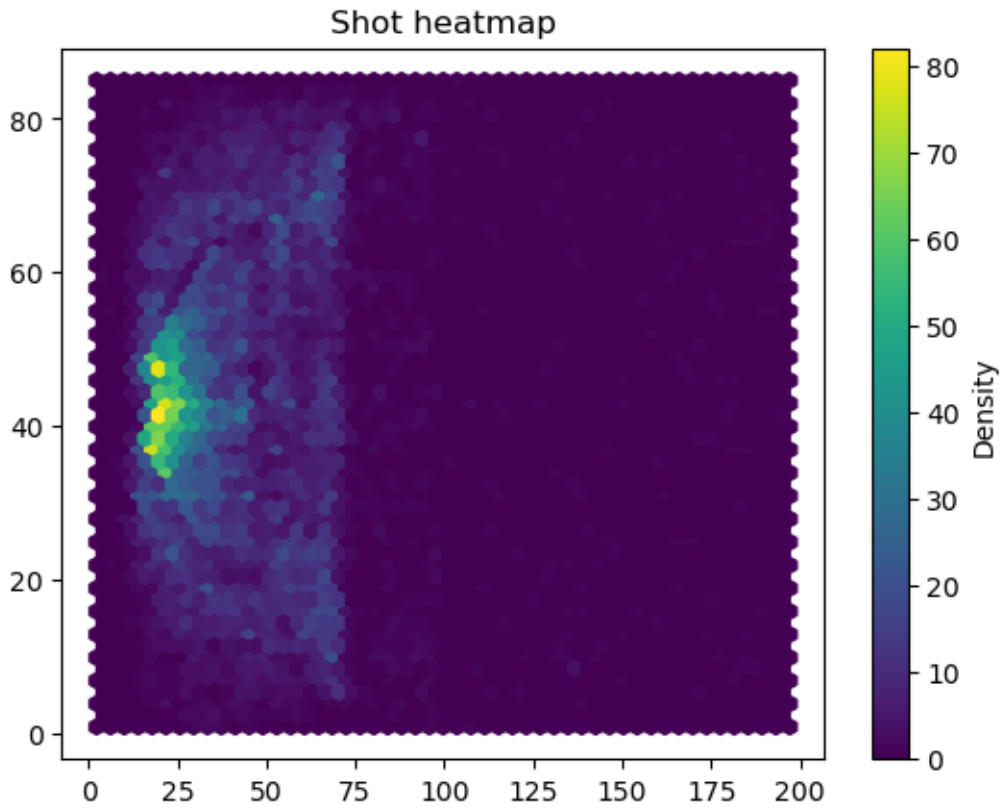
```
ax.add_patch(net)

plt.show()
```



[33]:
```
plt.hexbin(data=new, x='x_location', y='y_location', gridsize=50)

plt.colorbar(label='Density')
plt.title('Shot heatmap')
plt.show()
```



The two scatterplots above show the same data - the x_location and y_location of every event in the dataset (n=9431). With almost 10,000 observations, I chose to display this data with a heatmap as well to show the density of observations, especially around where the net would be (the yellow box in the first scatterplot).

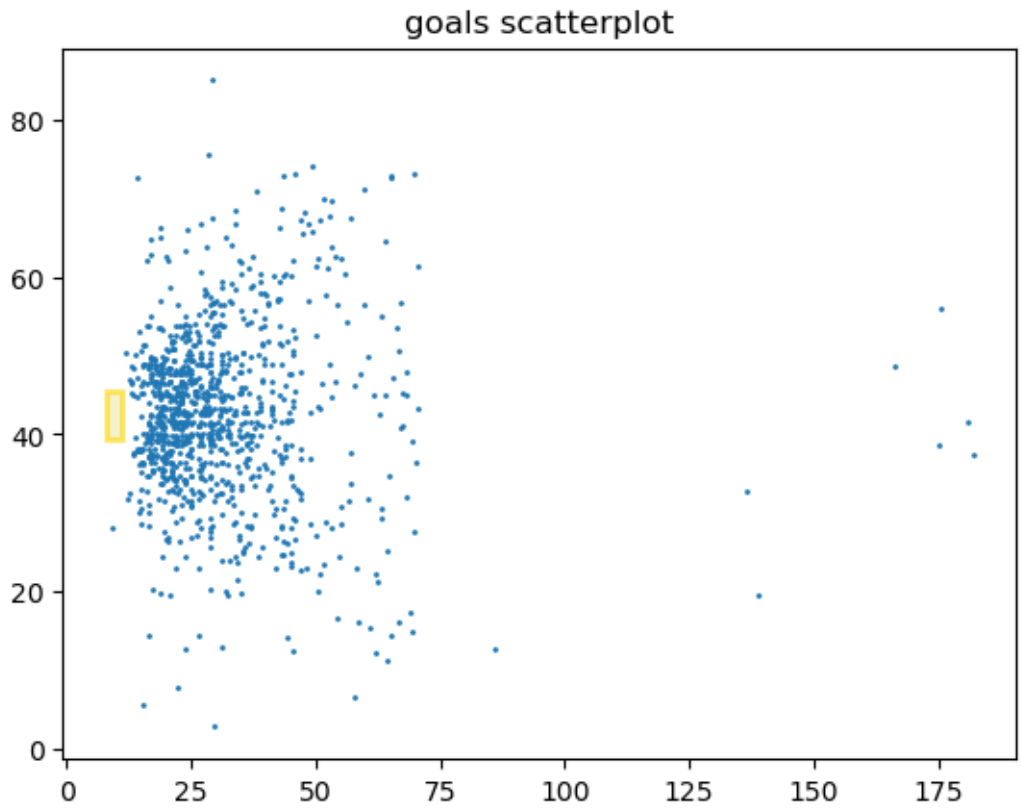
[36]:
```
goals_df = new[new['event'] == 'goal']

fig, ax = plt.subplots()

shot_scatter = ax.scatter(data=goals_df, x='x_location', y='y_location', s=1)

net = patches.Rectangle((8, 39.5), 3, 6, facecolor='palegoldenrod',␣
 ↪edgecolor='gold', linewidth=2, alpha=0.6)
ax.add_patch(net)

plt.title('goals scatterplot')
plt.show()
```



Correcting outlier goals Graphing just the goals, we can see that there are a few clear outliers outside of our defined offensive-zone range of 'x_location' = (0, 100). Because we removed empty net goals from our database, we can assume that these values were inputted/transformed incorrectly, as it is extremely unlikely that a goal would be scored on a goaltender from that far away. Upon reviewing the game footage (on the QMJHL Game Centres) for the seven goals in the table below, it was concluded that these goals did not happen this far away from the net. Therefore, we will transform these goals into values that fit our range of 'x_location' = (0, 100).

Shots from this far away will not be transformed, because it is possible and infrequently common for shots to be taken from this far away. We can see in the original event scatterplot above that these events are much less common than shots taken in the offensive zone. We will not consider these outliers to be discarded or adjusted because the volume is too high to verify each one on video, but we must adjust the goals inputted incorrectly to ensure the quality of the prediction models.

[9]:
```
new[(new['event'] == 'goal') & (new['x_location'] > 100)]
```

[9]:

	event	period_id	s	x_location	y_location	home	\
1223	goal	1	57	139.000000	19.550000	0.0	
2157	goal	2	1578	175.000000	38.533333	0.0	
2850	goal	1	397	175.333333	56.100000	0.0	

```
3318  goal          1   939  136.333333   32.866667   1.0
5792  goal          2  2199  182.000000   37.400000   0.0
5823  goal          1   855  181.000000   41.650000   1.0
8521  goal          1   756  166.333333   48.733333   0.0

            player_team_id goalie_team_id              player  \
1223               Moncton     Saint John        Shayne Gould
2157         Charlottetown  Victoriaville    Kimani Eccleston
2850         Rouyn-Noranda   Newfoundland    Charles Laforest
3318         Victoriaville     Shawinigan        Xavier Lebel
5792                Quebec  Victoriaville      Egan Beveridge
5823         Victoriaville     Sherbrooke  Thomas Desruisseaux
8521  Blainville-Boisbriand     Chicoutimi       Matt Gosselin

                    goalie  game_id  score_diff strength  s_since_prev  \
1223         Samuel Meloche    33070          -3      5v5           NaN
2157       Andrei Nikolayev    33203          -1      5v5          17.0
2850  Louis-Antoine Denault    32944           0      5v4         243.0
3318       Philippe Boucher    33204           0      5v5          55.0
5792    Anthony Catanzariti    33205           5      5v5          13.0
5823              Kyan Labbé    33206           0      5v4         213.0
8521        Patriks Plumins    32787           0      5v5           1.0

          x_prev     y_prev  distance_prev  shot_distance  shot_angle
1223         NaN        NaN            NaN     130.041157   10.164947
2157  174.000000  56.100000     149.311088     164.047964    1.385542
2850  135.333333  60.066667     114.971615     164.895132    4.730939
3318   71.333333  44.200000      65.980637     125.703005    4.395210
5792  183.333333  47.883333      10.567784     171.076036    1.708315
5823   34.333333  79.900000      39.636014     170.002125    0.286477
8521   27.333333  46.750000      12.247914     155.458351    2.297975
```

In the transformations below, 'x_location' must be transformed last because np.where() takes the "'x_location' >" condition to determine whether or not the other variables should be adjusted.

[10]:
```
import numpy as np

new['x_prev'] = np.where((new['event'] == 'goal').shift(-1) &␣
 ↪(new['x_location'] > 100).shift(-1), 200 - new['x_prev'], new['x_prev'])
new['y_prev'] = np.where((new['event'] == 'goal').shift(-1) &␣
 ↪(new['x_location'] > 100).shift(-1), 85 - new['y_prev'], new['y_prev'])
new['y_location'] = np.where((new['event'] == 'goal') & (new['x_location'] >␣
 ↪100), 85 - new['y_location'], new['y_location'])
new['x_location'] = np.where((new['event'] == 'goal') & (new['x_location'] >␣
 ↪100), 200 - new['x_location'], new['x_location'])
```

With these changes, we also need to recalculate distance_prev, shot_distance, and shot_angle.

[11]:
```
net_location = (11, 42.5)

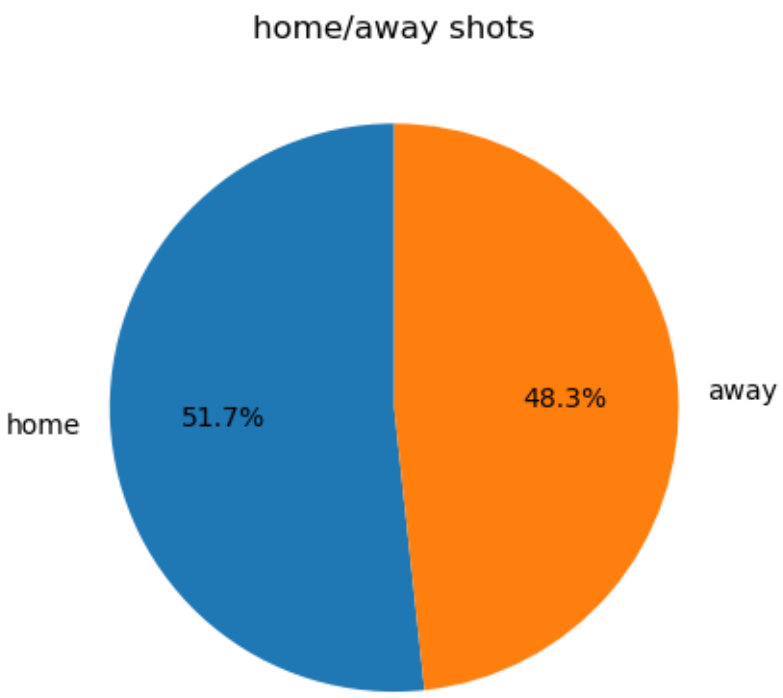
new['distance_prev'] = np.sqrt((new['x_location'] - new['x_prev']) ** 2 +␣
 ↪(new['y_location'] - new['y_prev']) ** 2)
new['shot_distance'] = np.sqrt((new['x_location'] - net_location[0]) ** 2 +␣
 ↪(new['y_location'] - net_location[1]) ** 2)
new['shot_angle'] = (np.degrees(np.arctan2((new['y_location'] -␣
 ↪net_location[1]), (new['x_location'] - net_location[0])))).abs()
```

[12]:
```
new.to_csv('2026-27 QMJHL PBP prepared.csv', index=False)
```

home

[41]:
```
home_counts = new['home'].value_counts()
home_pie = home_counts.plot.pie(
    autopct="%1.1f%%",
    startangle=90,
    labels=['home', 'away'],
    ylabel="",
)

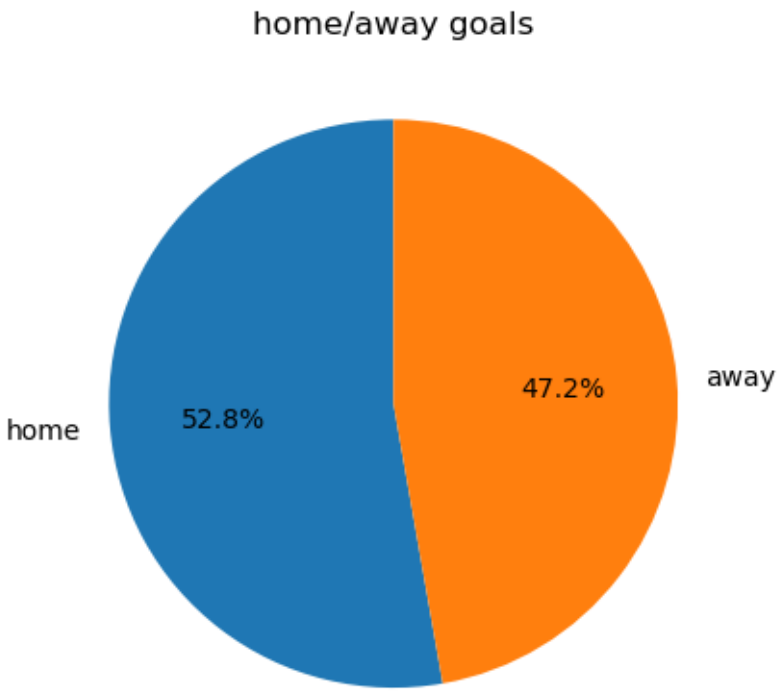
plt.title('home/away shots')
plt.show(home_pie)
```



As also shown in the 'mean' under 'home' in the DataFrame description table, the home team takes about 7% more shots than the away team in a game. "Home-ice advantage" and the 'home' binary classifier could potentially be a factor in the regression models.

[42]:
```
home_goal_counts = new['home'].where(new['event'] == 'goal').value_counts()
home_goal_pie = home_goal_counts.plot.pie(
    autopct="%1.1f%%",
    startangle=90,
    labels=['home', 'away'],
    ylabel="",
)

plt.title('home/away goals')
plt.show(home_goal_pie)
```

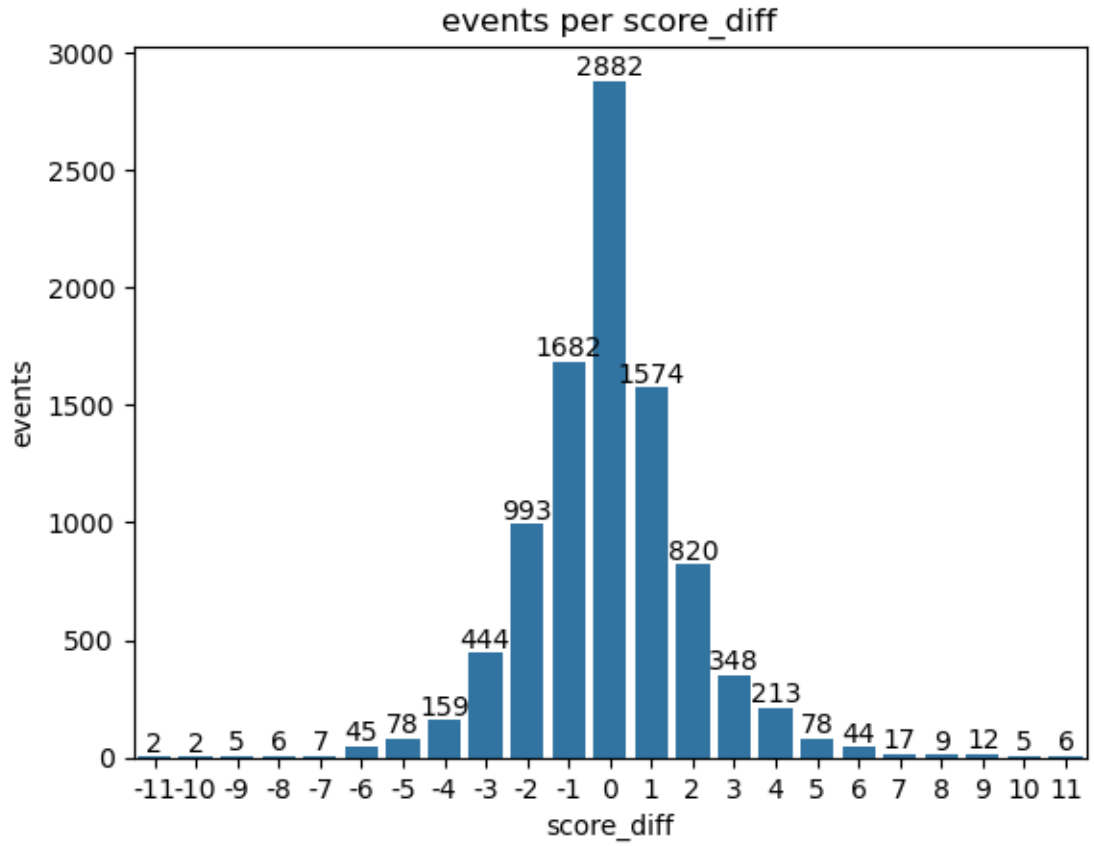


The home team has an even greater advantage in goal-scoring - the home team scores 52.8% of all goals in games. This is another indicator that the 'home' variable may play a significant role in the models.

score_diff

[43]:
```
score_count = sns.countplot(data=new, x='score_diff')
score_chart = score_count.bar_label(score_count.containers[0])
```

```
plt.title('events per score_diff')
plt.ylabel('events')
plt.show(score_chart)
```



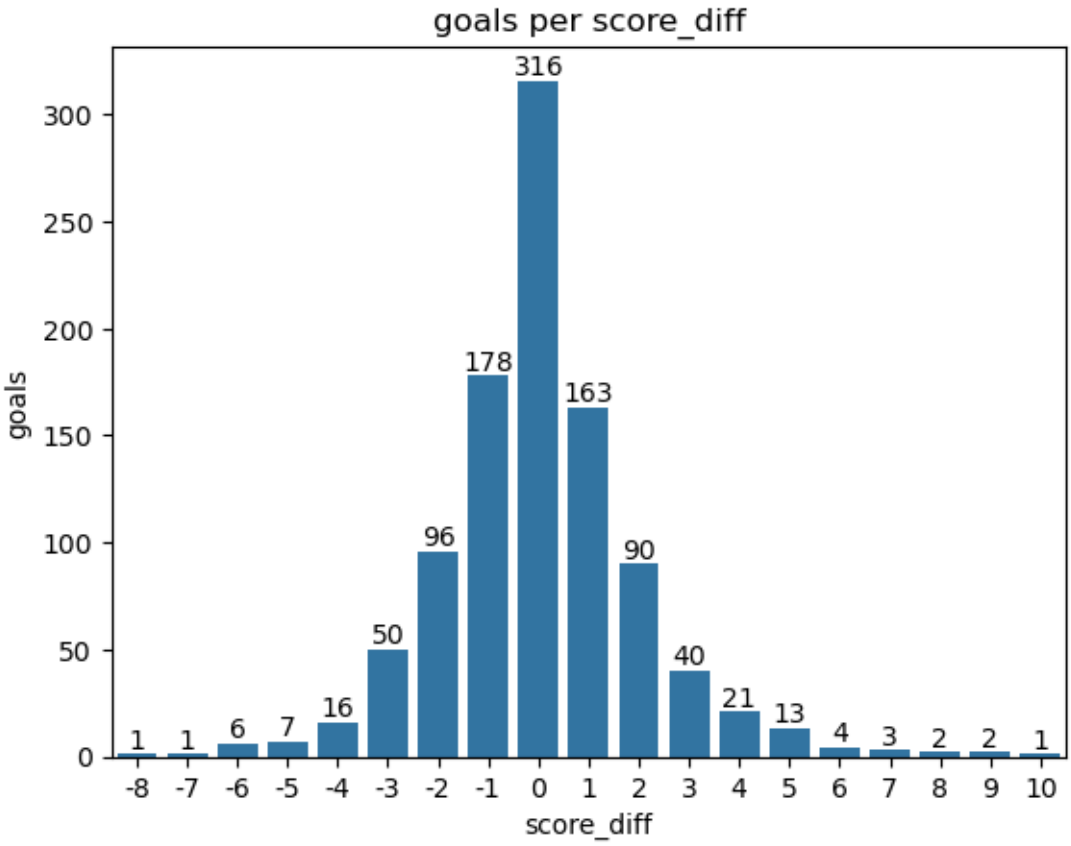
Graphing the frequency of events by score differential yields some interesting results. Of course, score_diff = 0 has the most events, as every game starts with this score differential by default. It is the one score_diff value that is guaranteed to be found in every game.

Looking at the positive-negative pairs (1 and -1, 2 and -2, etc.) can give us some information about 'score effects,' which has been explored at the NHL level and is part of why it is a variable in this QMJHL expected goals model. In the data, teams that were down by one, two, or three outshot the other team in each case. However, teams that were up by four outshot their losing opponent. Teams down by five were able to equal their opponents' shots on average, but this is where the sample starts to get very small. At the very high score differentials, the winning team began outshooting the losing team.

[44]:
```
score_goal_count = sns.countplot(data=goals_df, x='score_diff')
score_goal_chart = score_goal_count.bar_label(score_goal_count.containers[0])

plt.title('goals per score_diff')
plt.ylabel('goals')
```

```
plt.show(score_goal_chart)
```

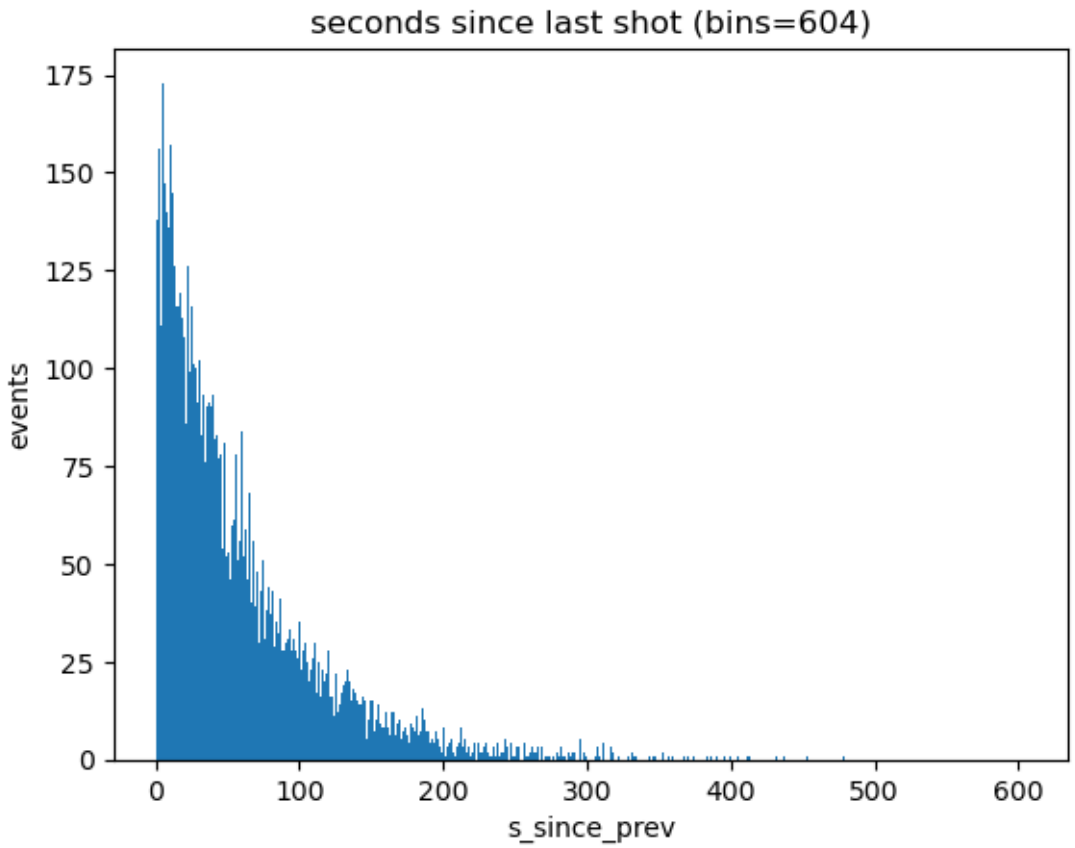


When we look at goals only, go-ahead goals are the most common (scored when the game is tied) for the same reasons as above - every game starts out tied. Atop the 161 goals that would be the first goals in the game included here, there are an additional 155 goals in the dataset that broke a tie score that was not 0-0 (1-1, 2-2, etc.). The second most common situation in the dataset is a game-tying goal, scored by a team down by one (score_diff = -1). Again, for teams down by one, two, or three, it was more common that they would score the next goal rather than the winning team.

s_since_prev

[45]:
```
s_since_chart = plt.hist(data=new, x='s_since_prev', bins=604)

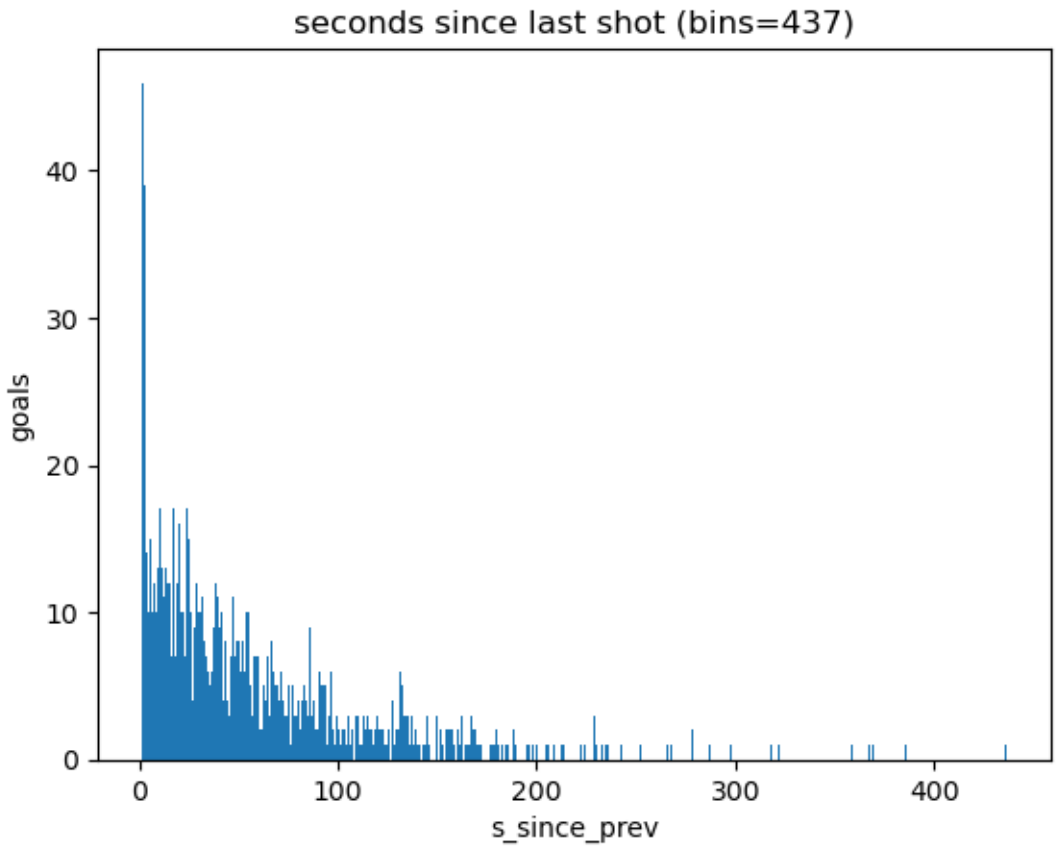
plt.title('seconds since last shot (bins=604)')
plt.ylabel('events')
plt.xlabel('s_since_prev')
plt.show(s_since_chart)
```



Graphing a histogram of the s_since_prev values by frequency, with 604 bins to represent the maximum s_since_prev value of 604. The single most common s_since_prev value appears to be = 4, with high frequencies all across the range from 1 to 10. It tapers off, drops off, then has another small peak at around s_since_prev = 50.

[46]:
```
s_since_goal_chart = plt.hist(data=goals_df, x='s_since_prev', bins=437)

plt.title('seconds since last shot (bins=437)')
plt.ylabel('goals')
plt.xlabel('s_since_prev')
plt.show(s_since_goal_chart)
```

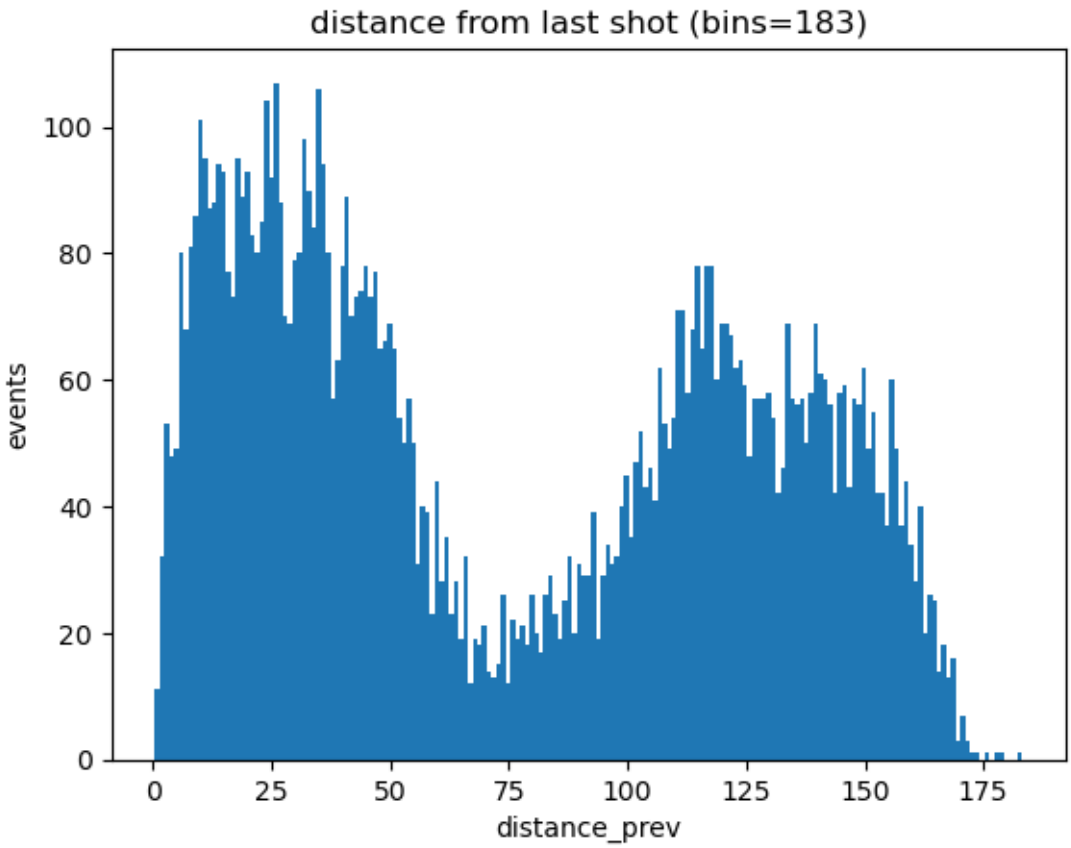


When we graph out just goals, the value of shots taken just 1 or 2 seconds after the previous shot increases greatly.

distance_prev

[52]:
```
prev_dist_chart = plt.hist(data=new, x='distance_prev', bins=183)

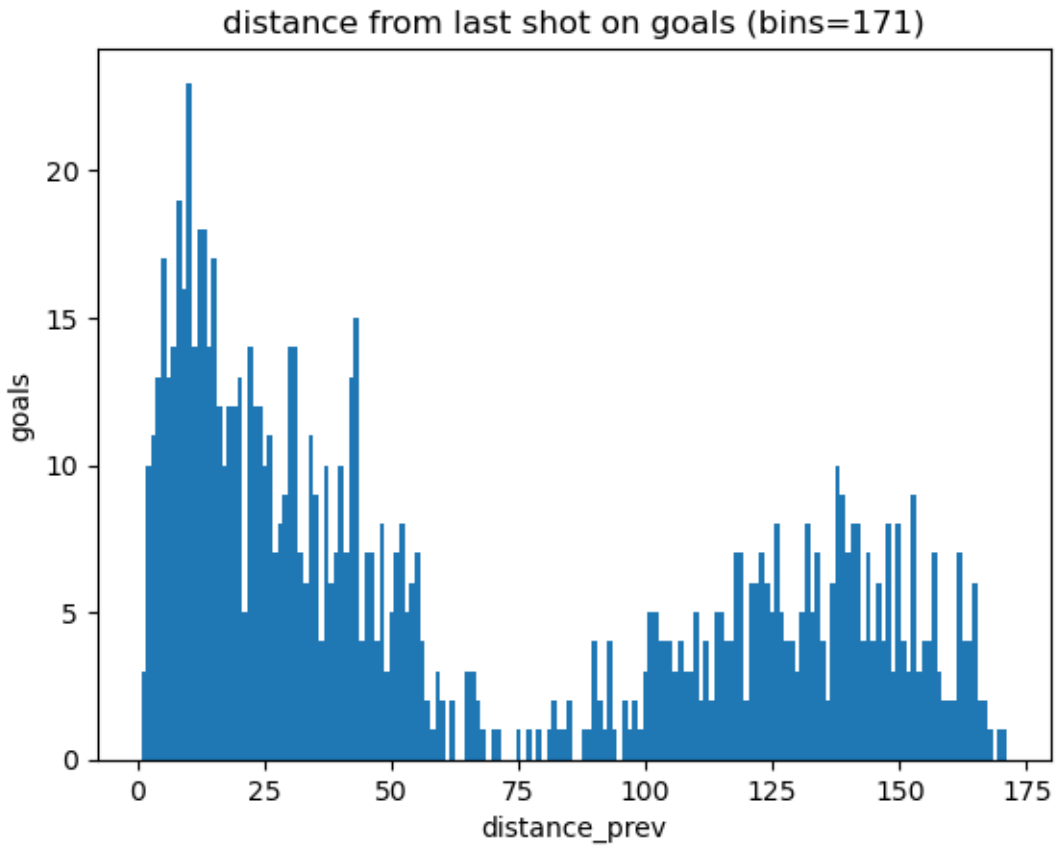
plt.title('distance from last shot (bins=183)')
plt.ylabel('events')
plt.xlabel('distance_prev')
plt.show(prev_dist_chart)
```



This data shows two peaks with a valley in between. Possibly, the first peak from around 0 to 50 ft came from shots taken sequentially by the same team, and the second smaller peak from about 100 to 150 ft came from shots taken by a team after the previous shot was taken by their opponents in the other end of the ice. The dip at around 50 to 100 ft perhaps represents the distance from typical shooting locations to the neutral zone (the middle of the ice, far away from both nets), where very few shots are taken.

[54]:
```
prev_dist_g_chart = plt.hist(data=goals_df, x='distance_prev', bins=171)

plt.title('distance from last shot on goals (bins=171)')
plt.ylabel('goals')
plt.xlabel('distance_prev')
plt.show(prev_dist_g_chart)
```

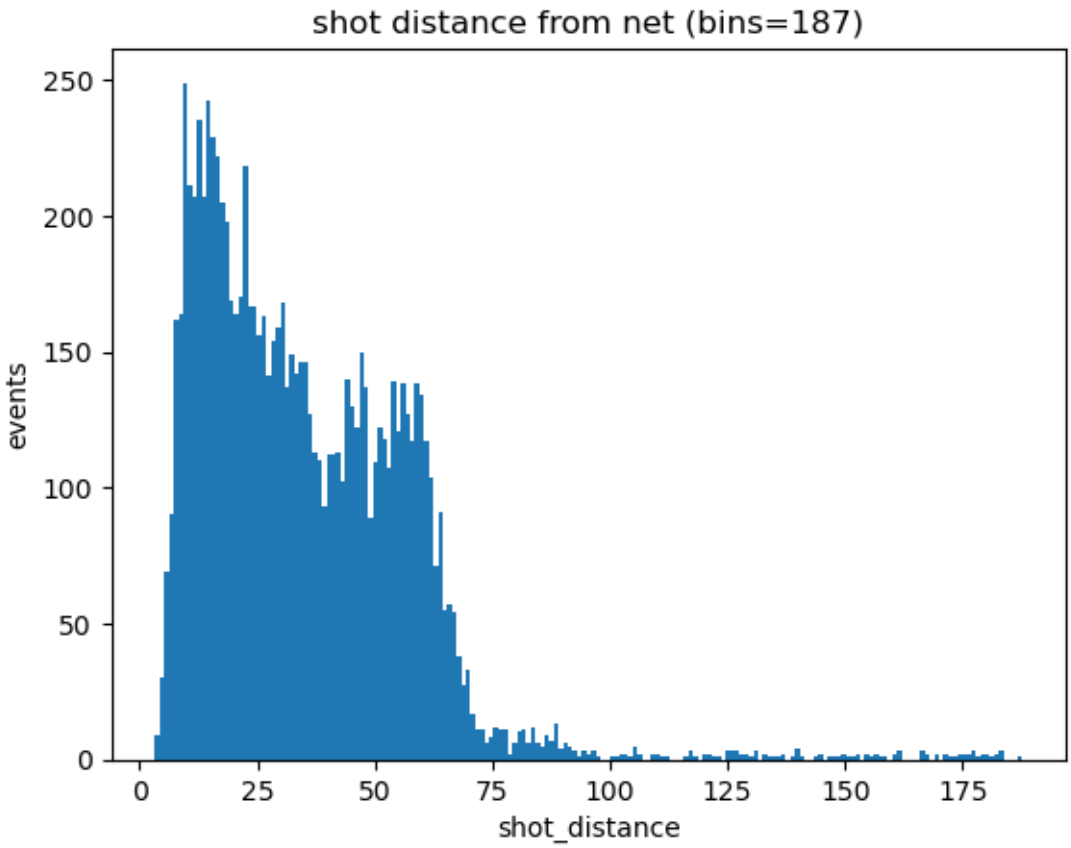


This pattern holds with the smaller sample of goals in the dataset, though the peak when the distance between shots is smaller is much higher relatively.

shot_distance

[56]:
```
distance_chart = plt.hist(data=new, x='shot_distance', bins=187)

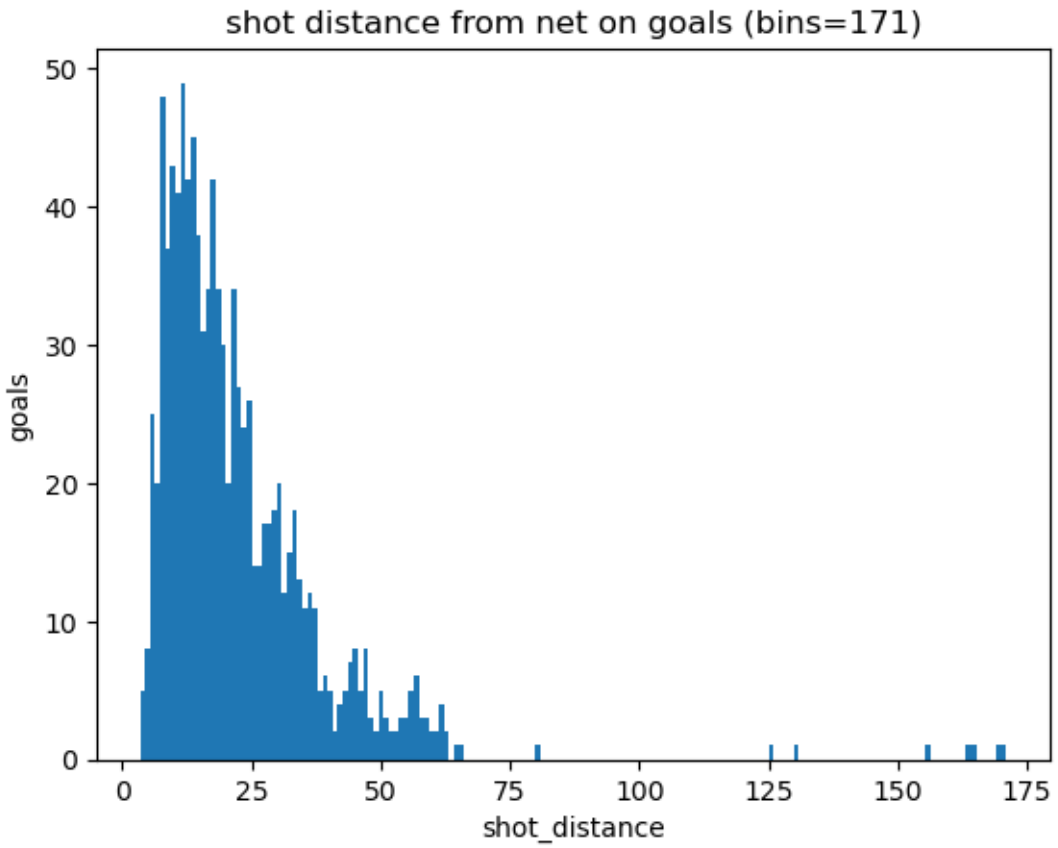
plt.title('shot distance from net (bins=187)')
plt.ylabel('events')
plt.xlabel('shot_distance')
plt.show(distance_chart)
```



The distance of shots from the middle of the net seems to fall most commonly between 5 and 20 ft, although up to around 65 ft is frequent. There are very few shots taken at a distance of over 75 ft away from the net.

[58]:
```
goals_distance_chart = plt.hist(data=goals_df, x='shot_distance', bins=171)

plt.title('shot distance from net on goals (bins=171)')
plt.ylabel('goals')
plt.xlabel('shot_distance')
plt.show(goals_distance_chart)
```

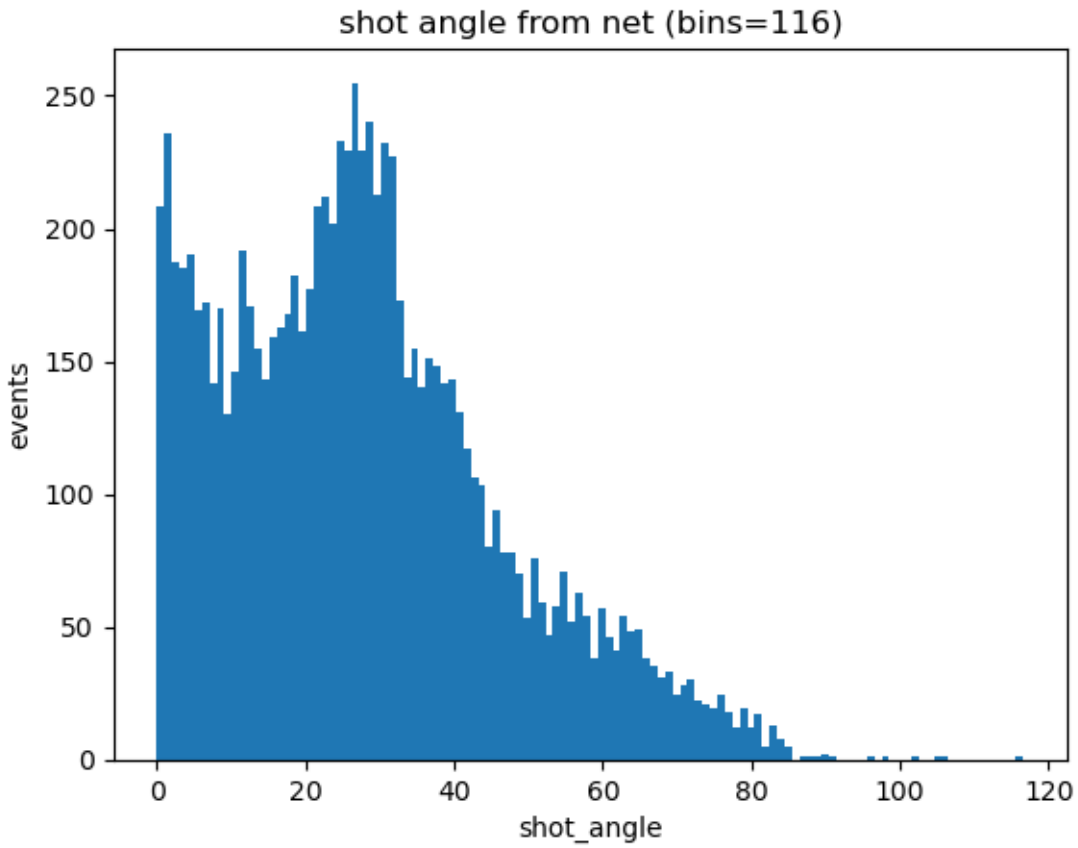


On goals, the results are a bit more evenly distributed between very close to the net (shot_distance $<$ 25 ft) and mid-range distances (such as between 25 and 40 ft).

shot_angle

[61]:
```
angle_chart = plt.hist(data=new, x='shot_angle', bins=116)

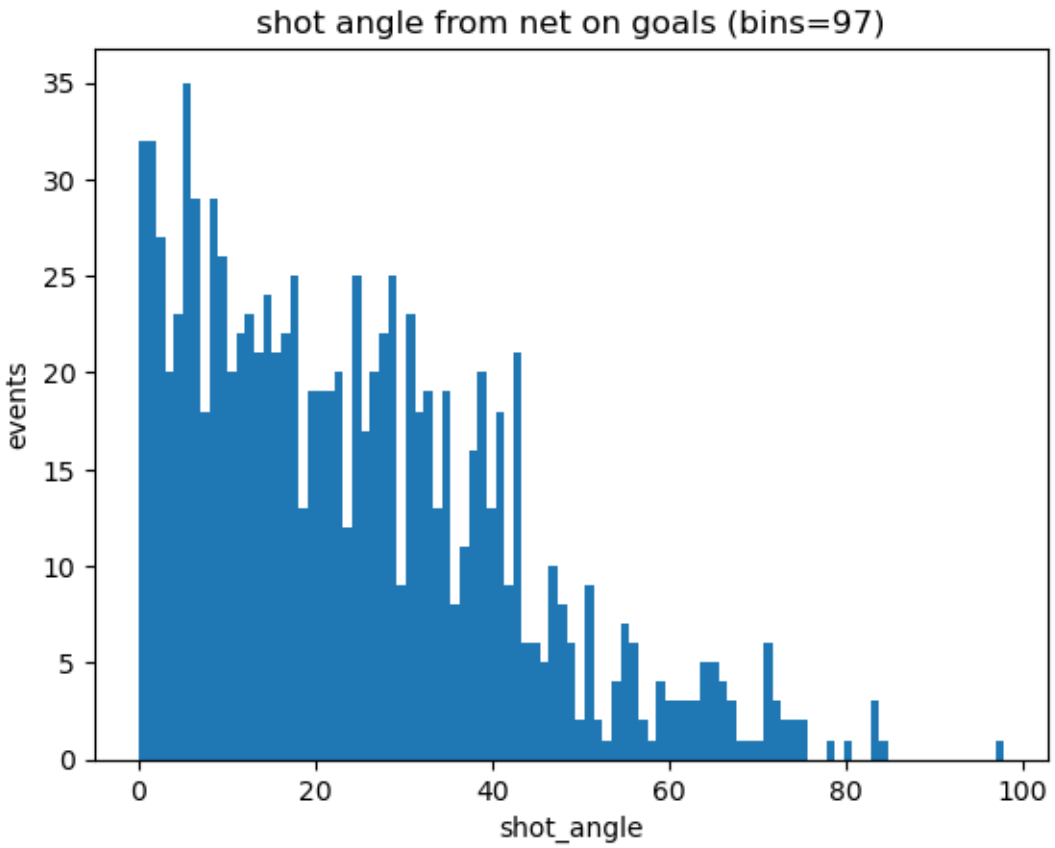
plt.title('shot angle from net (bins=116)')
plt.ylabel('events')
plt.xlabel('shot_angle')
plt.show(angle_chart)
```



For the angle of shots, it's very common to shoot directly in front of the net (where y_location = 42.5 ft) or at a slight angle between 20 and 30 degrees.

[ ]:
```
angle_goal_chart = plt.hist(data=goals_df, x='shot_angle', bins=97)

plt.title('shot angle from net on goals (bins=97)')
plt.ylabel('goals')
plt.xlabel('shot_angle')
plt.show(angle_goal_chart)
```



On goals, there are fewer and fewer goals in the dataset the more extreme the angle. This histogram suggests that the shot angle may be a significant variable in the regression models, with a higher incidence of goals the lower the shot angle is.

CIND820 Baseline Modeling

November 10, 2026

0.1 CIND820 Baseline Modeling

[51]:
```
import pandas as pd

model_prep = pd.read_csv('2026-27 QMJHL PBP prepared.csv')
```

0.1.1 Final preprocessing steps

One-hot encoding To one-hot encode the 'strength' categorical variable, we'll use OneHotEncoder from the sklearn.preprocessing package. This will create a binary variable for every strength in the dataset, like 'strength_5v5' or 'strength_3v3.' This follows how Younggren & Younggren encoded strength in their 2021 NHL expected goals model.

[52]:
```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
encoder.set_output(transform='pandas')

strength_encoded = encoder.fit_transform(model_prep[['strength']])
model_prep = pd.concat([model_prep, strength_encoded], axis=1)
```

To one-hot encode the 'score_diff' variable, extreme scores (up/down 4 or more) will be combined into a singular variable because of the rareness of the very high/low values, like +/-11. These are the thresholds used by Younggren & Younggren in their 2021 development of an NHL expected goals model.

[53]:
```
import numpy as np

model_prep['score_even'] = np.where(model_prep['score_diff'] == 0, 1, 0)
model_prep['score_up_1'] = np.where(model_prep['score_diff'] == 1, 1, 0)
model_prep['score_down_1'] = np.where(model_prep['score_diff'] == -1, 1, 0)
model_prep['score_up_2'] = np.where(model_prep['score_diff'] == 2, 1, 0)
model_prep['score_down_2'] = np.where(model_prep['score_diff'] == -2, 1, 0)
model_prep['score_up_3'] = np.where(model_prep['score_diff'] == 3, 1, 0)
model_prep['score_down_3'] = np.where(model_prep['score_diff'] == -3, 1, 0)
model_prep['score_up_4'] = np.where(model_prep['score_diff'] >= 4, 1, 0)
model_prep['score_down_4'] = np.where(model_prep['score_diff'] <= -4, 1, 0)
```

Transforming the target variable The target variable in these models will be named 'is_goal,' where 1 = a goal and 0 = not a goal.

[54]:
```
model_prep['is_goal'] = np.where(model_prep['event'] == 'goal', 1, 0)
```

Variable selection The code below transforms the model_prep DataFrame to include only the information and variables needed for the models. These variables are very similar to those outlined in Younggren & Younggren's NHL expected goals model.

[55]:
```
model_prep = model_prep[['game_id',
 'player_team_id', 'player',
 'goalie_team_id', 'goalie',
 'shot_distance',
 'shot_angle',
 's',
 'period_id',
 'x_location', 'y_location',
 'x_prev', 'y_prev',
 'distance_prev',
 's_since_prev',
 'home',
 'strength',
 'strength_3v3', 'strength_3v4', 'strength_3v5', 'strength_4v3',␣
 ↪'strength_4v4', 'strength_4v5', 'strength_5v3', 'strength_5v4',␣
 ↪'strength_5v5',
 'score_diff',
 'score_even', 'score_up_1', 'score_down_1', 'score_up_2', 'score_down_2',␣
 ↪'score_up_3', 'score_down_3', 'score_up_4', 'score_down_4',
 'is_goal']]
```

Split train and test sets The train set will be all games before October 19th, 2026 (100 games). The game_id of this final game before the cutoff was 33205. The code below returns the last chronological row in this game, of which its index will be where we split the data.

[56]:
```
model_prep[model_prep['game_id'] == 33205].iloc[-1:]
```

[56]:

	game_id	player_team_id	player	goalie_team_id	goalie	shot_distance	shot_angle	s	period_id	x_location	…	score_even	score_up_1	score_down_1	score_up_2	score_down_2	score_up_3	score_down_3	score_up_4	score_down_4	is_goal
5814	33205	Quebec	Charles-Antoine Dubé	Victoriaville	Anthony Catanzariti	11.28401	23.687526	3600	3	21.333333	…	0	0	0	0	0	0	0	1	0	0

```
[1 rows x 37 columns]
```

We will copy the model_prep DataFrame, containing only the variables and information needed in the model, to create new DataFrames accurately labeled as model_train and model_test.

[58]:
```
model_train = model_prep.loc[:5814].copy()
model_test = model_prep.loc[5815:].copy().reset_index()
```

0.2 Logistic regression

Data transformation: replacing NaN values In sklearn's linear_model package, the LogisticRegression class does not natively accept NaN values. These occur in our dataset when it comes to the first shot in every period, as 'x_prev,' 'y_prev,' 'distance_prev,' and 's_since_prev' are left blank because there was no shot preceding them. To provide a solution for this model, we will consider the faceoff at the beginning of the period as our previous event. This means that 'x_prev' and 'y_prev' will be 100 and 42.5 respectively (the location of the faceoff dot). The previous time in 's_since_prev' will be the beginning of the period.

For example, the very first row in our dataset has these values: * 's' = 86 * 'x_location' = 41 * 'y_location' = 21.5333333333333

In this case, the NaN rows will be populated: * 's_since_prev' = 86 * 'x_prev' = 100 * 'y_prev' = 42.5 * 'distance_prev' = 62.6147

[59]:
```
lr_prep = model_prep.copy()

lr_prep['x_prev'] = lr_prep['x_prev'].fillna(100)
lr_prep['y_prev'] = lr_prep['y_prev'].fillna(42.5)

lr_prep['s_since_prev'] = np.where((lr_prep['s_since_prev'].isna()) &␣
 ↪(lr_prep['period_id'] == 1), lr_prep['s'], lr_prep['s_since_prev'])
lr_prep['s_since_prev'] = np.where((lr_prep['s_since_prev'].isna()) &␣
 ↪(lr_prep['period_id'] == 2), (lr_prep['s'] - 1200), lr_prep['s_since_prev'])
lr_prep['s_since_prev'] = np.where((lr_prep['s_since_prev'].isna()) &␣
 ↪(lr_prep['period_id'] == 3), (lr_prep['s'] - 2400), lr_prep['s_since_prev'])
lr_prep['s_since_prev'] = np.where((lr_prep['s_since_prev'].isna()) &␣
 ↪(lr_prep['period_id'] == 4), (lr_prep['s'] - 3600), lr_prep['s_since_prev'])

lr_prep['distance_prev'] = np.sqrt((lr_prep['x_location'] - lr_prep['x_prev'])␣
 ↪** 2 + (lr_prep['y_location'] - lr_prep['y_prev']) ** 2)
```

Data grouping: strength situations Younggren & Younggren's NHL expected goals model makes the conscious choice to divide itself into three separate models based on 'strength' - we will call these three situations * even strength (EV): when both teams have the same number of players on the ice (5v5, 4v4, 3v3) * power play (PP): when the shooting team has more players on the ice than the other team because of a penalty (5v4, 5v3, 4v3) * penalty kill (PK): when the shooting team has fewer players on the ice than the other team because of a penalty (4v5, 3v5, 3v4)

[133]:
```
even_cols = ['strength_5v5', 'strength_4v4', 'strength_3v3']
lr_even = lr_prep[lr_prep[even_cols].eq(1).any(axis=1)]
pp_cols = ['strength_5v4', 'strength_5v3', 'strength_4v3']
lr_pp = lr_prep[lr_prep[pp_cols].eq(1).any(axis=1)]
pk_cols = ['strength_4v5', 'strength_3v5', 'strength_3v4']
lr_pk = lr_prep[lr_prep[pk_cols].eq(1).any(axis=1)]
```

0.2.1 Even strength (EV) logistic regression model

First, we will do the even strength logistic regression model since this is the 'default' game state and this state makes up the majority of playing time in most hockey games. From the features, we removed the Game ID, the target variable ('is_goal'), text variables (teams and players), as well as the 'strength' and 'score_diff' variables that our encoded variables were derived from. We are also dropping 'period_id' because it is already reflected with greater accuracy in the 's' (time elapsed in game) variable. Younggren & Younggren's analysis similarly drops the period as a numeric variable, which is why this decision was made.

[134]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, log_loss

features = lr_even.drop(columns=['game_id', 'is_goal', 'player_team_id',␣
 ↪'player', 'goalie_team_id', 'goalie', 'strength', 'score_diff', 'period_id'])
targets = lr_even['is_goal']

X_train = lr_even.loc[:5814].drop(columns=['game_id', 'is_goal',␣
 ↪'player_team_id', 'player', 'goalie_team_id', 'goalie', 'strength',␣
 ↪'score_diff', 'period_id'])
y_train = lr_even.loc[:5814, 'is_goal']

X_test = lr_even.loc[5815:].drop(columns=['game_id', 'is_goal',␣
 ↪'player_team_id', 'player', 'goalie_team_id', 'goalie', 'strength',␣
 ↪'score_diff', 'period_id'])
y_test = lr_even.loc[5815:, 'is_goal']

model_lr = LogisticRegression(max_iter=10000)
model_lr.fit(X_train, y_train)
```

[134]:
```
LogisticRegression(max_iter=10000)
```

The three evaluation metrics below were all used previously in the literature to evaluate NHL expected goals models - both AUC and log loss were displayed in Younggren & Younggren's 2021 model, while RMSE was mentioned in Sprigings & Toumi's 2015 work. All of these metrics for this model score around the 'fair/acceptable' range by these metrics.

[136]:
```
ytestpred = model_lr.predict_proba(X_test)[:, 1]

roc_auc = roc_auc_score(y_test, ytestpred)
print(f"ROC-AUC: {roc_auc:.3f}")
```

```
loss = log_loss(y_test, ytestpred)
print(f"Log loss: {loss:.4f}")

rmse = np.sqrt(np.mean((ytestpred - y_test) ** 2))
print(f"RMSE: {rmse:.4f}")
```

```
ROC-AUC: 0.738
Log loss: 0.2873
RMSE: 0.2874
```

Below are the coefficients of each feature in the model, listed in absolute order of magnitude from highest to lowest. The top of the list is dominated by the encoded strengths and score differentials, while shot distance is the strongest coefficient that does not relate to those two dimensions. Provided how Younggren & Younggren's model was dominated by more 'immediate' shot factors like shot distance and seconds since previous shot, it may be useful to train another logistic regression model on a set of features that excludes the strength and score differential features.

It would be interesting to compare this model to the more expansive model above, to see whether a shot's immediate circumstances (such as its location and its timing) are better predictors than the existing state of the game that cannot be changed by a single shot. The second strongest coefficient is 'score_down_4,' and its negative value means that the expected goals on a shot by a team that is down by four or more goals is greatly diminished by this circumstance. I believe that this shows up so extremely because teams that are losing by four or more goals are often vastly outmatched by their opponents and struggle greatly to score.

These score effects are perhaps much more emphasized in the QMJHL as compared to the NHL because poor-performing teams are seen as part of the "junior hockey cycle" - because players can only play a maximum of five years in the league, there is a lot of turnover year-by-year and teams tend to "accept" poor-performing seasons in order to get better draft picks and select players who will help the team win in 2-4 years. Having these score differentials in the logistic regression model may blur the lines and place an outsized importance on the teams that players are on, rather than evaluating the true quality of their shot.

This could cause considerable problems when answering Question 2, where we want to find the top xG generators on poor-performing teams. This could cause us to overlook a player who is on a team that loses by four or more goals often, where their xG is greatly diminished by their team's situation.

[135]:
```
coef_table = pd.DataFrame({
    'Feature': features.columns,
    'Coefficient': model_lr.coef_[0]
})

coef_table = coef_table.sort_values(by='Coefficient', key=abs, ascending=False)

coef_table
```

[135]:

	Feature	Coefficient
18	strength_5v5	-0.481756
27	score_down_4	-0.451805
14	strength_4v4	-0.254622
21	score_down_1	-0.227530
22	score_up_2	-0.178778
25	score_down_3	0.166019
20	score_up_1	-0.160218
0	shot_distance	-0.147693
9	home	0.126280
10	strength_3v3	-0.118994
19	score_even	-0.116951
3	x_location	0.098332
26	score_up_4	0.075158
23	score_down_2	0.058050
24	score_up_3	-0.019317
5	x_prev	0.007646
7	distance_prev	-0.007083
1	shot_angle	-0.003567
6	y_prev	-0.001247
4	y_location	0.000784
8	s_since_prev	0.000747
2	s	-0.000011
12	strength_3v5	0.000000
13	strength_4v3	0.000000
16	strength_5v3	0.000000
17	strength_5v4	0.000000
11	strength_3v4	0.000000
15	strength_4v5	0.000000

The code below adds a variable named 'xg_lr' to the lr_even DataFrame. This variable contains the predicted probabilities from the logistic regression model. This is what expected goals are - a single shot's probability of becoming a goal, on a scale from 0 to 1.

[138]:
```
lr_even['xg_lr'] = model_lr.predict_proba(features)[:, 1]
```

```
/var/folders/0n/31k33nxn59sgvdqjp51nfl2c0000gn/T/ipykernel_3295/559152845.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  lr_even['xg_lr'] = model_lr.predict_proba(features)[:, 1]
```

Below are the top ten shots by xG in the training set. We can see that the maximum xG value is ~0.38. These values are relatively low because the approximate naive probability of a shot becoming a goal is 10% (assuming a .900 average save percentage across the league). Of these top ten 'most valuable' shots in the training set, three of them were goals.

[141]:
```
lr_even[['strength', 'player_team_id', 'player', 'score_diff',␣
 ↪'goalie_team_id', 'shot_distance', 'shot_angle', 's_since_prev', 'is_goal',␣
 ↪'xg_lr']].loc[:5814].sort_values(by='xg_lr', ascending=False).head(10)
```

[141]:

	strength	player_team_id	player	score_diff	goalie_team_id	\
4060	4v4	Newfoundland	Benjamin Veitch	-2	Rimouski	
5595	5v5	Rimouski	Jérémy Turcot	-3	Chicoutimi	
4254	3v3	Halifax	Shawn Carrier	0	Drummondville	
3820	5v5	Gatineau	Gabriel Séguin	-3	Baie-Comeau	
1661	5v5	Rimouski	Rafaël Cloutier	10	Shawinigan	
5443	5v5	Shawinigan	Gleb Olkhovik	4	Charlottetown	
3556	5v5	Charlottetown	Cameron Haye	0	Drummondville	
3555	4v4	Charlottetown	Kage Flory	0	Drummondville	
3581	5v5	Drummondville	Yoan Tassé	-2	Charlottetown	
123	3v3	Cape Breton	Noah Jettelson	0	Halifax	

	shot_distance	shot_angle	s_since_prev	is_goal	xg_lr
4060	6.557523	15.025222	104.0	0	0.375929
5595	5.916174	16.699244	16.0	0	0.364095
4254	9.118053	18.103763	27.0	0	0.357212
3820	8.740455	7.450249	21.0	1	0.346355
1661	8.045030	6.064922	110.0	1	0.344367
5443	7.666667	0.000000	105.0	1	0.344218
3556	6.006686	2.703626	194.0	0	0.342134
3555	6.762314	9.648045	56.0	0	0.340974
3581	5.071735	9.648045	2.0	0	0.339533
123	9.769923	8.337444	44.0	0	0.332645

Below are the top ten shots by xG in the test set, as predicted by the logistic regression model. The maximum xG value here is also ~0.38. Interesting, three of these top ten shots were also goals.

[115]:
```
lr_even[['strength', 'player_team_id', 'player', 'score_diff',␣
 ↪'goalie_team_id', 'shot_distance', 'shot_angle', 's_since_prev', 'is_goal',␣
 ↪'xg_lr']].loc[5815:].sort_values(by='xg_lr', ascending=False).head(10)
```

[115]:

	strength	player_team_id	player	score_diff	\
8022	5v5	Sherbrooke	Thomas Rousseau	-3	
8207	3v3	Moncton	Liam Kilfoil	0	
6534	5v5	Halifax	Mateo Nobert	4	
8905	5v5	Charlottetown	Matthew Butler	0	
6540	5v5	Halifax	Oleg Kulebiakin	4	
9227	5v5	Gatineau	Zakary Horvat-Edouard	-3	
8873	5v5	Halifax	Patrick McNab	-2	
7118	5v5	Newfoundland	Benjamin Veitch	-2	
6614	3v3	Newfoundland	Louis-François Bélanger	0	
7969	5v5	Quebec	Mavrick Rousseau-Hamel	-2	

	goalie_team_id	shot_distance	shot_angle	s_since_prev	is_goal	\

8022	Halifax	5.730062	8.530766	12.0	1
8207	Newfoundland	8.045030	6.064922	126.0	0
6534	Charlottetown	6.672685	2.433606	145.0	0
8905	Cape Breton	5.400643	9.055358	304.0	0
6540	Charlottetown	9.703965	5.025152	107.0	0
9227	Val-d'Or	6.880003	14.305552	2.0	1
8873	Saint John	8.376571	5.824028	6.0	0
7118	Charlottetown	10.700480	4.556130	158.0	1
6614	Victoriaville	11.600299	18.513044	54.0	0
7969	Baie-Comeau	7.041464	18.778033	39.0	0

	xg_lr
8022	0.379583
8207	0.376683
6534	0.359747
8905	0.344871
6540	0.332317
9227	0.330424
8873	0.329023
7118	0.326429
6614	0.324880
7969	0.323104

In the next steps, I will explore other models (such as XGBoost) and also build a version of the best-performing model without the strength and score differential attributes, primarily to answer Question 2 (evaluating players' individual performances).